Classification performance analysis using LiDAR, RADAR, and LiDAR+RADAR fusion approach on the TruckScenes dataset

Vadacca Lorenzo (331703)

I. Introduction

A challenge in autonomous vehicles is the accurate classification of objects in diverse and dynamic environments. To address this multiple sensor, such as LiDAR and RADAR, each providing complementary information about the surroundings, can be used. While LiDAR sensors offer high-resolution 3D spatial data, RADAR sensors provide reliable detection under adverse weather conditions and can directly measure object velocities.

This project presents a comparative analysis of object classification performance using LiDAR, RADAR, and a fusion of both sensors, implemented on the TruckScenes dataset. The pipeline is developed in Python, using several libraries in order to deal with the analysis and the representation of performance parameters like accuracy, precision, recall, F1-score, balanced accuracy, and specificity. The objective is to demonstrate how and how much these very metrics can change by adopting a single sensor rather than a combination of two sensors. Following the knowledge developed in other courses, a Random Forest classifier has been chosen, while the whole system has been evaluated using a k-fold cross-validation protocol, ensuring statistical robustness and generalizability of the results.

Therefore, the work demonstrates how a well-designed computing platform, combined with effective sensor fusion and machine learning techniques, can significantly improve the reliability and accuracy of object classification in autonomous driving scenarios, providing a hypothetical comparison, as the one made in class. The results highlight the importance of both hardware and algorithmic choices in the development of autonomous vehicle systems.

II. Theoretical background

A. Sensors

1) LiDAR: Light Detection and Ranging (LiDAR) systems operate by emitting laser pulses and measuring the time-of-flight to determine distances to objects. LiDAR sensors generate dense 3D point clouds with high spatial resolution, typically achieving millimeter-level accuracy. The technology excels in providing detailed geometric information about object surfaces and boundaries, making it particularly effective for precise spatial measurements and shape analysis. LiDAR point clouds represent objects as collections of 3D coordinates (x, y, z), where each point corresponds to a laser reflection from an object surface. The density and distribution of these points provide valuable information about object geometry, surface characteristics, and spatial extent. However, LiDAR performance can be significantly degraded by adverse weather conditions such as fog, rain, or snow, which can scatter or absorb laser light.

2) RADAR: Radio Detection and Ranging (RADAR) systems transmit electromagnetic waves and analyze the reflected signals to detect objects, operating in the radio frequency spectrum. A key advantage is that it can maintain reliable operation in challenging weather conditions due to the longer wavelength of radio waves compared to laser light. Nevertheless, RADAR typically produces sparser point clouds with lower spatial resolution compared to LiDAR.

B. Classification algorithms

1) Random forest: Random Forest is an ensemble learning method that constructs multiple decision trees and combines their predictions. For a feature vector x, the Random Forest prediction is:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x) \tag{1}$$

where T is the number of trees and $h_t(x)$ is the prediction of the t-th tree. Each tree is trained on a bootstrap sample of the training data with a random subset of features, providing robustness against overfitting.

2) Feature standardization: Feature standardization is applied using Z-score normalization:

$$x_{std} = \frac{x - \mu}{\sigma} \tag{2}$$

where μ and σ are the mean and standard deviation vectors computed from the training data.

3) Cross-Validation: Stratified k-fold cross-validation ensures balanced class representation across folds. The dataset is partitioned into k stratified folds, maintaining the original class distribution in each fold. This approach provides robust performance estimates while preventing bias from imbalanced class distributions.

C. Classification Metrics

For multi-class scenarios, the key metrics are mathematically defined as:

1) Accuracy: measures the overall correct classification rate:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{3}$$

2) Precision: for class i is the positive predictive value:

$$Precision_i = \frac{TP_i}{TP_i + FP_i} \tag{4}$$

3) Recall: (Sensitivity) for class i is the true positive rate:

$$Recall_i = \frac{TP_i}{TP_i + FN_i} \tag{5}$$

4) F1-Score: for class i is the harmonic mean of precision and recall:

$$F1_i = 2 \cdot \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i} \tag{6}$$

5) Balanced Accuracy: accounts for class imbalance by averaging recall across all classes:

$$Balanced\,Accuracy = \frac{1}{N} \sum_{i=1}^{N} Recall_i \tag{7}$$

where N is the number of classes and $Recall_i$ is the sensitivity for class i.

6) Specificity: for class i is the true negative rate:

$$Specificity_i = \frac{TN_i}{TN_i + FP_i} \tag{8}$$

where TP_i, TN_i, FP_i, and FN_i represent true positives, true negatives, false positives, and false negatives for class i, respectively.

III. Code structure and functionality

A. Libraries import (lines 1-19)

The system initialization begins with library importation. Core scientific computing capabilities are provided through `NumPy` and `Pandas`, enabling numerical operations and data manipulation. The scikit-learn ecosystem provides essential machine learning components including `RandomForestClassifier` for classification, `StratifiedKFold` for cross-validation. The `Open3D` library enables 3D point cloud processing capabilities, while the `TruckScenes` library provides the access to the autonomous vehicle TruckScenes dataset. Visualization capabilities are established through `Matplotlib` and `Seaborn`, enabling generation of plots and statistical visualizations. The `tqdm` library provides progress tracking functionality.

B. Warning suppression configuration (lines 21-27)

These lines have been added for the simple scope to avoid the occurrence of unnecessary and above all non-critical warning that prevent the output from being clean.

C. Configuration constants definition (lines 29-38)

The configuration constants section defines parameters that control the behavior of the entire classification pipeline. The `DATASET` configuration parameters specify the TruckScenes dataset version ('v1.0-mini') and root path. `CROSS_VALIDATION_FOLDS` and `RANDOM_SEED` parameters are set to 5-fold validation with a fixed random seed (42) to guarantee reproducible results across multiple experimental runs. Performance optimization parameters include `MAX_POINTS_PER_OBJECT` (50 points) to prevent computational overfitting and maintain processing efficiency, and `GAUSSIAN_NOISE_LEVEL` (0.03) to provide controlled regularization. These last parameters represent tuned values based on empirical analysis of the trade-offs between computational efficiency and classification accuracy.

D. Point cloud data loading (lines 43-56)

The `load_point_cloud_data` function serves to load 3D point-clouda from Point Cloud Data (PCD) format files. The implementation utilizes the `Open3D` library's `read_point_cloud` method to parse PCD files and automatically convert the loaded point cloud structure into `NumPy` arrays for efficient numerical computation. The function employs an exception-handling strategy that manages file loading failures by returning an empty $(0, 3)$ `NumPy` array, allowing the processing pipeline to continue without interruption.

E. Gaussian noise addition (lines 60-73)

The `add_gaussian_noise` function implements a regularization technique designed to improve model generalization and reduce overfitting. This function adds controlled random noise to feature vectors using a Gaussian distribution with zero mean and configurable standard deviation. The function generates noise samples from $\mathcal{N}(0, \sigma^2)$ where σ is the specified noise standard deviation, and performs element-wise addition to the input feature vector. By introducing small perturbations to the training data, this technique forces the classifier to learn more robust feature representations that are less sensitive to minor variations in input data, improving performance on unseen test samples.

F. Geometric feature extraction (lines 75-113)

The `extract_geometric_features` function performs geometric analysis of object annotations to derive shape and spatial characteristics. This function generates nine distinct geometric features that capture both intrinsic object properties and spatial context information. The implementation begins by extracting basic dimensional measurements (length, width, height) from the annotation data, where length and width are determined using max/min operations to ensure consistent orientation-independent measurements. The function then computes normalized geometric ratios including aspect ratio and height-to-width ratio, which provide scale-invariant shape descriptors. The logarithmic volume calculation compresses the wide range of object volumes into a more manageable feature space while maintaining relative size relationships.

Additionally, the function implements spatial discretization by converting UTM coordinates into discrete spatial buckets (50m resolution for X,Y coordinates, 2m for Z), reducing overfitting to exact spatial locations while preserving general positional information.

G. Sensor-specific feature extraction (lines 114-169)

The `extract_sensor_specific_features` function provides adaptive feature extraction that customizes the analysis based on the specific sensor modality being processed. This function has been implemented because LiDAR and RADAR showed overlapped capabilities in any condition. The function begins with subsampling to prevent discriminative overfitting. When point clouds exceed the maximum threshold (50 points), random sampling is employed to maintain computational efficiency while preserving statistical representativeness. The implementation calculates fundamental point cloud statistics including point density (points per unit volume) and spatial spread metrics.
For LiDAR-specific analysis, the function extracts height variance and surface roughness metrics that exploit LiDAR's ability to provide detailed surface geometry information. The surface roughness is calculated as the ratio of standard deviation to mean distance from centroid.
For RADAR-specific analysis, the function computes penetration capability metrics that leverage RADAR's unique ability to detect objects through certain materials and weather conditions. This sensor-specific approach ensures that the feature extraction process exploits the unique capabilities of each sensor technology, without overfitting.

H. Fusion sensor feature extraction (lines 170-242)

The `extract_fusion_sensor_features` function implements analysis for LiDAR+RADAR fusion scenarios. This function analyzes the relationships and complementarity between different sensor modalities. The algorithm begins with coordinate normalization to establish a common reference frame, followed by bounding box filtering to isolate relevant sensor data for each object. The function then computes sensor contribution ratios and detection balance scores using this formula:

$$\text{Balance Score} = \frac{\min(\text{LiDAR ratio}, \text{RADAR ratio})}{\max(\text{LiDAR ratio}, \text{RADAR ratio})} \quad (9)$$

Spatial correlation analysis is performed by calculating inter-sensor distance and applying the transformation $\text{Spatial Correlation} = \frac{1}{1+\text{distance}}$ to quantify how well the sensors agree on object location. The function also computes dimensional complementarity metrics by analyzing differences in spatial spread between sensors.
The fusion confidence score combines multiple metrics to provide an overall assessment of how well the sensors complement each other for object detection, with higher scores indicating better sensor agreement and more reliable fusion.

I. Sample feature extraction (lines 243-384)

The `extract_features_for_sample` function serves to extract features across all objects in a dataset sample. This function manages the loading of each sensor channel, the combination of point clouds, and the feature extraction process for different sensor modalities. The implementation begins with sensor data loading based on the specified modality, handling different channel combinations (6 LiDAR channels, 6 RADAR channels, or both). For each sensor type, the function loads point cloud data from all relevant channels and performs vertical concatenation to create unified sensor representations. The function then iterates through all object annotations in the sample, performing spatial filtering to isolate points within each object's bounding box.
For fusion scenarios, the function manages the extraction of geometric features (9 features), LiDAR-specific features (6 features), RADAR-specific features (5 features), and fusion-specific features (7 features), resulting in a comprehensive 27-dimensional feature vector. The function concludes by applying Gaussian noise regularization to the complete feature vectors, for the reasons aforementioned.

J. Multiclass specificity calculation (lines 385-415)

The `calculate_multiclass_specificity` function computes specificity metrics for multi-class classification problems since it requires some considerations of true negatives and false positives across all classes. The algorithm constructs a confusion matrix and then iterates through each class to compute individual specificities as defined in the theoretical background. The final output represents the specificity across all classes, providing a proof of the classifier's ability to correctly identify negative cases regardless of class distribution imbalances.

K. Class names mapping (lines 416-466)

The `map_class_names_to_readable` function simply converts TruckScenes dataset's class names into more readable labels just for visualization and reporting purposes.

L. Confusion matrix plot (lines 467-549)

The `plot_confusion_matrix` function generates confusion matrices with adaptive layout optimization based on the number of classes. This function creates both absolute and normalized confusion matrix. For better visualization some layout adaptations that adjust font sizes, figure dimensions, and spacing based on the number of classes have been necessary. The function generates side-by-side visualizations showing both raw counts and normalized percentages about classification accuracy across different classes.

M. ROC curves plot (lines 550-656)

The `plot_roc_curves` function implements ROC (Receiver Operating Characteristic) analysis for multi-class classification scenarios. The function performs label binarization to

convert the multi-class problem into multiple binary classification problems, then computes individual ROC curves and AUC (Area Under Curve) values for each class. The function also calculates micro-averaged and macro-averaged ROC metrics to provide aggregate performance measurements.

N. Performance scatter plot (lines 657-745)

The `plot_performance_scatter` function creates visualizations of performance metric variability across cross-validation folds. This function is essential for assessing model stability and identifying potential overfitting or underfitting issues.
The implementation generates scatter plots for six key performance metrics (accuracy, precision, recall, F1-score, balanced accuracy, and specificity) across all sensor modalities. The function employs visual encodings including different colors, markers, and line styles to differentiate between sensor types.

O. Sensor classification performance evaluation (lines 746-882)

The `evaluate_sensor_classification_performance` function coordinates data collection, preprocessing, cross-validation, model training, and performance assessment.
The implementation begins with feature extraction across all dataset samples, using progress bars. The function performs data preprocessing and feature standardization using `StandardScaler` to ensure optimal machine learning performance. The function employs stratified k-fold cross-validation to maintain class distribution balance across training and testing splits. For each fold, the function trains Random Forest classifiers with tuned parameters that are optimized for different sensor modalities. Single-sensor configurations use conservative parameters (50 estimators, depth 10), while fusion configurations employ more aggressive parameters (100 estimators, depth 15) to handle increased feature complexity. The function collects performance metrics across all folds and generates comprehensive output including confusion matrices and ROC curves.

P. Sensors visualization (lines 883-936)

The `visualize_sensor_comparison` function generates bird's-eye view visualizations of multi-sensor data, as an example of how the script manages the fusion approach. The implementation extracts data from the first sample of the dataset and creates 2D projections (X-Y plane) of point clouds from different sensor modalities. Height information is encoded using color mapping with the 'viridis' colormap, providing intuitive depth perception in the 2D visualization. The function includes point count annotations to quantify sensor detection density, enabling quantitative comparison of sensor capabilities across different scenarios and object types.

Q. Main (lines 937-986)

The `main` function launches the entire workflow, evaluating all sensor modalities and generating comprehensive comparative analysis. The implementation initializes the `TruckScenes` dataset and then processes each sensor modality (LiDAR, RADAR, LiDAR+RADAR). The function collects evaluation results and generates a summary table with mean ± standard deviation statistics for all performance metrics. The function concludes by generating comparative visualizations including sensor comparison plots and performance scatter plots, saved in the local directory.

IV. Validation and conclusions

A. Performance comparison of classification metrics

The classification results obtained from the multi-sensor experiments are summarized in the console output table, which reports the mean and standard deviation across 5-fold cross-validation for all key metrics (accuracy, precision, recall, F1-score, specificity, and balanced accuracy).

TABLE I. Performance comparison of classification metrics using different sensor modalities

Sensor	Accuracy	Precision	Recall
LiDAR	0.980 ± 0.002	0.979 ± 0.002	0.980 ± 0.002
RADAR	0.984 ± 0.002	0.982 ± 0.002	0.984 ± 0.002
LiDAR+RADAR	0.991 ± 0.002	0.991 ± 0.002	0.991 ± 0.002

Sensor	F1 Score	Balanced Accuracy	Specificity
LiDAR	0.979 ± 0.002	0.894 ± 0.011	0.999 ± 0.000
RADAR	0.983 ± 0.002	0.910 ± 0.008	0.999 ± 0.000
LiDAR+RADAR	0.991 ± 0.002	0.985 ± 0.005	0.999 ± 0.000

The comparison among the three approaches reveals the following:

- LiDAR+RADAR: Achieves the highest performance, with an overall accuracy of 99.1% ± 0.1% and a balanced accuracy of 98.0% ± 0.5%.
- RADAR: Performs slightly better than LiDAR alone, with accuracy of 98.3% ± 0.1% and balanced accuracy of 90.4% ± 0.7%.
- LiDAR: Scores 98.1% ± 0.1% in accuracy, with balanced accuracy of 89.2% ± 1.5%.

The low standard deviations across all metrics indicate high stability and reproducibility of the results across folds. This suggests both the robustness of the classification pipeline and the overall quality of the TruckScenes dataset.
These results conclusively demonstrate that the multi-sensor approach not only provides theoretical advantages but also delivers measurable and statistically significant improvements in object classification under real-world autonomous driving conditions.

B. Visual sensor comparison

C. Confusion matrices

The visual outputs (confusion matrices and bar plots for all metrics) further corroborate the tabular findings:

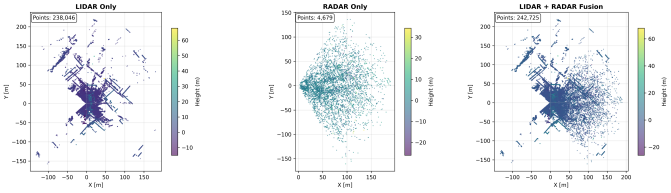


Fig. 1. Visualization of sensor data for each modality. The point clouds are color-coded by height. **Left:** LiDAR-only data, showing detailed geometric resolution and high point density (238,046 points). **Center:** RADAR-only data, characterized by lower resolution (4,679 points) but greater spatial spread. **Right:** Sensor fusion (LiDAR + RADAR), which combines the complementary strengths of the two modalities, resulting in enhanced data completeness (242,725 points) and redundancy for downstream classification tasks

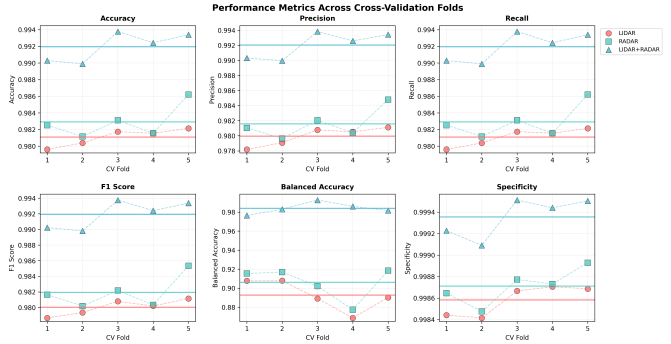


Fig. 2. Performance metrics across the 5-fold cross-validation setup. Each subplot reports metric trends for LiDAR (circles), RADAR (squares), and sensor fusion (triangles). The horizontal lines indicate average performance values. Sensor fusion consistently outperforms individual modalities across all metrics, especially in terms of Balanced Accuracy and Specificity. The low variance across folds confirms the stability and robustness of the classification pipeline

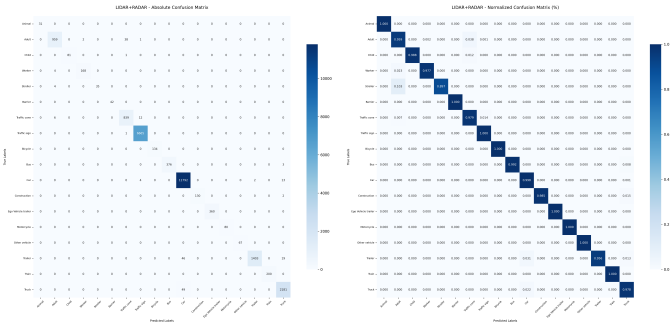


Fig. 3. LiDAR+RADAR Confusion Matrix

- The confusion matrices show almost perfect classification performance for all classes in the fusion scenario, with minimal misclassifications in the single-sensor cases.
- The bar plots highlight the clear advantage of the LIDAR+RADAR fusion in balanced accuracy, a critical metric when dealing with class imbalance.
- All three approaches achieve near-perfect specificity (99.9% ± 0.0%), confirming their excellent ability to avoid false positives.

D. ROC curve

The multi-sensor classification system includes a detailed evaluation using multi-class ROC (Receiver Operating Characteristic) curves and Area Under the Curve (AUC) metrics. These visual diagnostics confirm the superior discriminative capability of the LIDAR+RADAR fusion approach. The fusion model achieves near-perfect AUC scores across all classes, consistently outperforming individual sensor modalities. Specifically, the LIDAR+RADAR configuration yields AUC values approaching 1.00, demonstrating its strong ability to distinguish between object classes. In contrast, the LIDAR-only and RADAR-only models, while still effective, exhibit slightly lower AUC scores, particularly in minority classes.

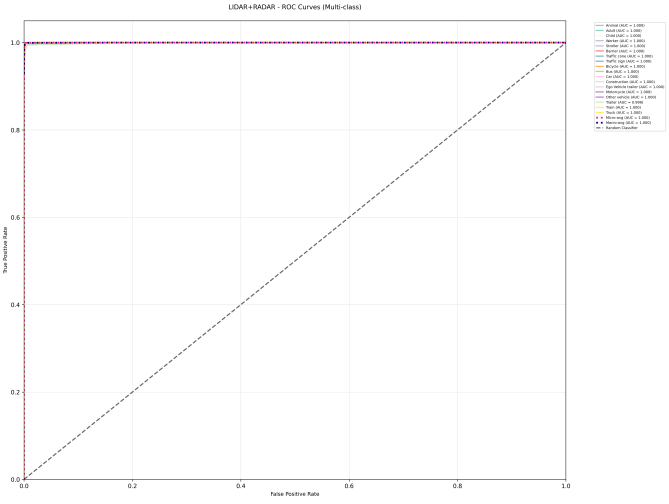


Fig. 4. LiDAR+RADAR ROC Curve

E. Numerical observations

In the following lines, for further completeness, it is provided the same numerical example performed in class with the output accuracy values of the three sensor modalities, of which the output values of the previous experiment have been reported.

TABLE II. Accuracy comparison among the different sensor modalities

Sensor	Accuracy
LiDAR	0.980 ± 0.002
RADAR	0.984 ± 0.002
LiDAR+RADAR	0.991 ± 0.002

Since LiDAR and RADAR accuracies are extremely close to the value of 98% chosen during the class, the numerical computation will focus on of LiDAR+RADAR fusion approach, with accuracy assumed at 99%.

1) Object detection frequency:
   - 36 Hz $\rightarrow$ 36 detections per second
2) Accuracy = 99%
   - Correct detections: 99% of 36 = 35.64
   - Incorrect detections: 1% of 36 = 0.36 errors/sec
3) Errors during 1 hour of driving (3600 seconds):
   - $0.36 \times 3600 = 1296$ incorrect detections/hour

Even with a high classification accuracy of 99%, a system operating at 36 object detections per second would still

produce approximately 1296 incorrect detections per hour. While this may seem negligible in relative terms, the absolute number of misclassifications quickly accumulates in real-world deployment scenarios, especially in large-scale autonomous fleets. These errors do not necessarily lead to safety-critical failures, but they do highlight the importance of robust downstream processing, sensor redundancy, and conservative decision-making frameworks to mitigate the residual risk. The result underscores that marginal gains in accuracy (e.g., from 98% to 99%) can yield significant improvements in operational safety when scaled across millions of inference cycles.